SayangCare: A Fault-Tolerant Stateful Voice Session Manager for Distributed Telehealth

Darren Tan Jun Le, Pieter Imanuel, Chew How Way, Jerlyn Ho, Eng Li Qi, Lucas Ee, Goh Yi Xuan
CSC3104 Cloud and Distributed Computing — AY2026/27, Team 25

Abstract— Telephone-based support lowers interface barriers but depends on stateful, distributed services. SayangCare combines Rust/Actix Web, signed Twilio webhooks, Redis Sentinel, PostgreSQL, distress-aware circuit breaking, and volunteer escalation. An automatic Redis failover initially lost 17 acknowledged session creations. The revised design commits versioned PostgreSQL checkpoints before acknowledgment and treats Redis as a disposable projection. A repeat 20-VU, 120-second fault test completed 2,149 lifecycles without HTTP errors or lost acknowledged operations. Awaited graceful API shutdown enabled a 2,090-lifecycle pod-loss run without HTTP errors. Local PostgreSQL shutdown tuning reduced observed recovery from 61 to 28 seconds, but 1.600% HTTP failures remained. Ten real Groq turns completed sentiment and reply operations without audio fallback, with median caller-turn latency 1.13 seconds. The workspace passed 100 tests; a PostgreSQL durability test was executed separately. Experiments use synthetic callers on single-node Minikube, not production or multi-node evidence. Live Twilio validation remains outstanding. SayangCare is a technical prototype, not a clinical or crisis-response service.

Keywords— distributed systems, voice interface, session recovery, circuit breaker, Raft, telehealth triage

1. Introduction and Motivation

Access to digital support is not equally convenient for everyone. A telephone call does not require a user to install an application, navigate a touchscreen, or own a modern smartphone. Voice interfaces and social robots have consequently been investigated as ways to support older adults and people experiencing isolation [7], [8]. That evidence concerns user engagement and wellbeing outcomes; it does not establish that an AI system can provide clinical care. SayangCare therefore treats its conversational component as a prototype for supportive listening and deterministic risk triage, not diagnosis, treatment, or an emergency service.

Telephone access changes the engineering problem. A call may span multiple webhook requests, be processed by different API instances, and depend on a third-party speech and telephony provider and an external inference API. Losing one process must not erase the conversation state, and a slow inference dependency must not hold every caller indefinitely. A conventional stateless request/response service does not by itself solve either issue. Callers at elevated risk also need a path to a human operator rather than an unbounded conversation with a model.

The submitted proposal set out two linked ideas: preserve context across service failures and adapt circuit-breaker behaviour to distress. The implementation develops these using versioned durable checkpoints, a Redis hot projection, an inactivity sweeper, cached-audio degradation, and auditable volunteer assignments. Failure testing motivated a change from Redis-first acknowledgment to PostgreSQL-backed commit authority. The project also integrates Twilio webhooks and provides Docker and Kubernetes deployment artefacts.

The contribution is an application-specific integration, not a new consensus protocol or validated clinical triage algorithm. The design connects caller state, inference degradation, and human escalation. Synthetic fault tests measure committed-state preservation and bounded fallback; they do not establish real-call safety or human outcomes.

2. Related Work

The circuit-breaker pattern isolates a failing dependency by stopping calls after observed failures and allowing a controlled recovery probe [1], [2]. This limits cascading failure and wasted work, but a conventional breaker treats requests according to service health rather than the consequence of failure to a particular user. SayangCare retains the familiar Closed, Open, and Half-Open states, while its adaptive policy lowers error-rate and latency limits as the caller's distress signal increases. The intended effect is to choose a conservative fallback sooner for a caller who may need timely human support. Unlike general-purpose resilience libraries, the policy is connected to a caller/session risk signal; unlike a generic audio fallback, it is designed to preserve the conversation and create an escalation path. Its heuristic signal and thresholds are prototype choices and require validation before operational use.

Raft provides a consensus-based approach to replicated state machines [3]. SayangCare uses the OpenRaft library to replicate queue commands for enqueue, claim, transfer, and release operations. The project does not propose a new Raft algorithm: its engineering contribution is applying a replicated priority queue to human handoff while keeping relational assignment history in PostgreSQL. This separates replicated queue coordination from durable audit records. The Redis sorted-set queue remains the default backend, making the Raft configuration optional rather than a prerequisite for local operation.

Redis Sentinel monitors instances and promotes a replica after master failure [4], but asynchronous replication is not a durable commit protocol. SayangCare's initial active-call design exposed this distinction experimentally. The revised design persists acknowledged active state in synchronously replicated PostgreSQL, retaining Redis as a best-effort projection. Terminal archival precedes version-safe cleanup. This protects tested acknowledged operations without claiming linearizability across all dependencies or arbitrary partitions.

Voice assistants and socially assistive systems have been studied for loneliness and social support, including voice-only modalities [7], [8]. Such work motivates an accessible interaction mode but evaluates human outcomes that SayangCare has not measured. The system focuses instead on distributed reliability and a human-review handoff. It does not claim the benefits reported by those studies, provide a dedicated always-on robot, or demonstrate that its risk classifier is clinically accurate. The distinction is material: SayangCare adds failure-handling and coordination mechanisms around a voice workflow, while prior intervention studies investigate whether a particular device or interaction benefits users.

3. System Design

SayangCare is structured as a modular monolith in a Rust Cargo workspace. The `core` crate defines domain models and service ports; infrastructure crates implement persistence, telephony, queueing, and circuit breaking; the Actix Web API composes the components and exposes HTTP endpoints. This keeps deployment to one API service while making the interfaces between major responsibilities explicit.

Figure 1. Request and state flow:

```
Caller <-> Twilio <-> Actix API
                      |-- PostgreSQL: versioned commit authority (CAS)
                      |-- Redis Sentinel: best-effort hot projection
                      |-- Groq inference: distress-aware breaker
                      |      `-- cached audio / deterministic reply
                      |-- Redis queue or OpenRaft: high-risk handoff
                      `-- PostgreSQL: archived calls and assignments
```

3.1 Call and state lifecycle

Twilio sends form-encoded requests to `/twilio/voice`, `/twilio/gather`, and `/twilio/status`; SayangCare returns TwiML rather than JSON on these routes. An incoming `CallSid` becomes the stable session identifier, so webhook retries can resolve to the existing session instead of creating a new conversation. The `/gather` handler records the speech transcript, obtains a response, and returns another gather prompt. The status callback drives terminal archival. Separate JSON endpoints

support the local Call Lab, session inspection, and volunteer operations.

Each active-session mutation commits a monotonically versioned PostgreSQL checkpoint before acknowledgment. SQL compare-and-swap (CAS) prevents a stale request replacing a newer transcript. Reads and CAS use checkpoint authority, not the potentially stale Redis projection. Projection failures are logged and bounded to 100 ms. Completion commits terminal state, archives the JSONB transcript, and retires only the matching version from active recovery. The checkpoint remains authoritative after archival. The sweeper scans unarchived checkpoints every 30 seconds, recovering terminal sessions and abandoning calls inactive for five minutes even if Redis lost their keys. Legacy active Redis sessions can be adopted lazily during upgrade.

3.2 Inference degradation and risk handling

For non-crisis turns, the API can call a Groq-compatible chat-completion service. Inference calls pass through a sliding-window circuit breaker that records latency and failures, estimates the 95th-percentile latency, and uses Closed, Open, and Half-Open states. A single recovery probe is admitted during Half-Open. Request generations prevent stale completions from changing the state of a newer breaker generation. An adaptive policy scales the error and latency limits according to a normalized distress signal; when inference is unavailable, the service uses a deterministic text reply or a bundled holding-audio response.

Risk classification is a local, weighted phrase heuristic assigning levels 1–5. Levels 4–5 meet the immediate-escalation condition. A high-risk turn bypasses generative inference, receives a fixed safety-oriented response, and is queued for human review. Thus the model is not responsible for deciding whether to persist the session or whether to enqueue a high-risk turn. This reduces dependence on the inference provider for the escalation path, but a phrase matcher can miss paraphrases, context, negation, and multilingual speech. It must not be interpreted as a validated mental-health assessment.

3.3 Queue, assignments, and deployment

The queue interface has two backends. Redis sorted-set queueing is default; optional OpenRaft replicates commands and orders items by descending risk, enqueue time, and identifier. Queue request identifiers make command retries idempotent. PostgreSQL stores assignment history and prevents conflicting active bindings. Operator alerts also use shared PostgreSQL storage and survive API replacement; the volunteer roster remains process-local. Queue and session mutations are not one cross-system transaction.

Docker Compose supplies local dependencies; Kubernetes supplies API Deployment/Service, Redis/Sentinel StatefulSets, PVCs, HPA and NGINX Ingress. The evaluated single-node Minikube cluster uses three API replicas, three Redis nodes, three Sentinels, and a three-instance CloudNativePG cluster with synchronous replica acknowledgment. Prometheus scraping, Adapter request-rate scaling and seven Grafana panels were verified locally. API database pools are limited to three connections per process to respect the primary's connection budget. Public DNS/TLS, live Twilio routing, node/zone failure and production deployment remain unvalidated.

4. Implementation

The workspace separates seven crates: `api`, `core`, `session-store`, `circuit-breaker`, `priority-queue`, `telephony`, and `redis-client`. Actix Web applies request tracing, compression, and request identifiers; it exposes the Call Lab dashboard and JSON routes for session and volunteer workflows. The Twilio middleware reconstructs the externally visible URL, sorts form parameters, and verifies the `X-Twilio-Signature` HMAC using the configured auth token. Missing or invalid signatures are rejected. Twilio integration includes Basic-Auth REST calls for call updates and outbound creation, status-callback registration, conference/redirect TwiML, and a signed `/twilio/outbound-connect` route. Real credentials and a public HTTPS callback are required for live calls.

The session-store crate uses SQLx migrations, including `0010_session_checkpoints.sql`, for durable versioned snapshots and cold archives. PostgreSQL acquisition is bounded to two seconds. Redis retains serialized projections with a one-hour TTL, but is not the session commit authority. Archive failures retain checkpoints for retry; completion is not fabricated on a failed database commit. Terminal checkpoints currently have no automatic purge policy, increasing storage cost. Concurrent Redis projection writes may be stale; authoritative reads prevent that cache state replacing a committed transcript.

The circuit-breaker crate maintains shared Redis-backed bounded samples, calculates error rate and p95 latency, trips after three consecutive failures, and atomically reserves one recovery probe across API replicas. Sentiment scoring and reply generation have separate breaker state, so scorer failure does not mark a successful reply as failed. Sentiment only adjusts operational degradation thresholds; deterministic local triage remains responsible for escalation. This signal and the risk heuristic are not clinically validated and must not be treated as a mental-health assessment. Holding audio is a versioned local WAV asset served by the API.

The priority-queue crate defines the `PriorityQueue` abstraction, Redis implementation, and OpenRaft state machine and transport. The Raft backend supports peer HTTP requests, leader forwarding, snapshots, request-id deduplication, and optional local persistence. It must be explicitly enabled and configured with peer membership; Redis remains the default. Volunteer assignment changes are also written to PostgreSQL, which provides durable history independently of queue membership.

Twilio outbound REST calls now have a ten-second timeout, explicit response errors, SID validation and six local contract/guard tests; these are not live TEL-06 evidence. GPT-OSS inference uses low reasoning effort, hidden reasoning, a 512-token output budget and JSON sentiment output; other models omit reasoning-specific fields. Metrics expose route latency, stage outcomes, pool and breaker gauges; metrics scrapes refresh shared breaker state with a bounded timeout. A stale coordination read returns 503 rather than success-shaped metrics.

5. Evaluation

On 26 October 2026 (Singapore time), `cargo test --workspace --locked` passed 100 tests with five environment-dependent tests ignored. The registered PostgreSQL checkpoint integration test was separately executed and passed: it verifies durable writes with unavailable Redis, competing CAS, stale-write rejection, archive fencing and failed database commits. A new registered shutdown test verifies an in-flight response completes during graceful termination. Eight offline Python evidence-gate tests passed. Native-process runner regression checks assert both exit 0 and 99 with separate logs. Raft leader-loss tests are in-process simulations, not Kubernetes node-failure evidence.

All workloads used synthetic callers on single-node Minikube. k6 incoming/turn/hangup/read-back checks validate IDs and transcript contents; dependency tests independently audit SQL records against stage-specific acknowledged IDs.

Experiment	Workload / measured result
Earlier Redis-first baseline	100 VUs / 30 s; 2,142 lifecycles; zero HTTP failures; HTTP p95/p99 390.04/514.97 ms
Earlier API-pod loss	100 VUs / 60 s; 3,901 lifecycles; zero HTTP failures; replacement Ready after 46.68 s
Earlier timed HPA	100 VUs / 120 s; 8,418 lifecycles; desired 3 to 9; first increase 67.0 s; additional Ready 79.6 s; return to 3 after 493.9 s cooldown
Repeatable durable baseline	100 VUs / 30 s; 1,358 lifecycles; zero failed checks or HTTP requests; HTTP p95/p99

Experiment	Workload / measured result
	997.87/1,155.75 ms; native exit 0
API pod loss after graceful-shutdown fix	100 VUs / 60 s; 2,090 lifecycles; zero HTTP failures; replacement Ready 10.112 s; native exit 0
Final checkpoint HPA evaluation	100 VUs / 120 s; 3,421 iterations; replicas 3 to 5 to 3; seven HTTP failures (0.0512%); HTTP p95/p99 2,200.26/3,706.21 ms; native exit 99
Automatic Redis promotion after fix	20 VUs / 120 s; 2,149 lifecycles; zero HTTP failures; promotion/readiness 8.031 s
PostgreSQL replacement, local shutdown refinement	20 VUs / 120 s; 93.895% lifecycle success; 1.600% HTTP failures; promotion/readiness 27.943 s; native exit 99
Real Groq, ten sequential turns	Sentiment 10/10, replies 10/10, no audio fallback; median 1,126.83 ms, max 1,443.03 ms

The Redis-first automatic promotion test lost 17 of 1,597 acknowledged incoming IDs, absent from SQL and all three Redis nodes. This failed result motivated durable checkpointing; it is retained, not reclassified as a passing experiment. The new automatic pause/promotion test preserved all 2,149 acknowledged creates, turns and hangups. PostgreSQL replacement preserved all 1,715 acknowledged creates, 1,700 turns and 1,693 hangups, but k6 exited 99: recovery consistency passed while availability failed. A later sweep archived all 1,716 committed checkpoints (1,698 completed, 18 abandoned), leaving none pending.

Independent SQL auditing verified all 1,358 baseline and 2,090 pod-loss archives, exact caller/assistant transcripts, matching checkpoints and zero pending records. API read-back may read a checkpoint, so it does not substitute for SQL archive auditing. The earlier 1,160-lifecycle baseline and original HPA run lacked captured exit codes; those historical nulls are not retroactively replaced. The new .NET-based runner captures native process completion. A first timed post-checkpoint HPA rerun measured 3 to 5 to 3 replicas, desired increase 84.8 s, additional Ready 108.7 s and downscale 383.1 s after load; its 3,791 iterations included five failed HTTP requests, so native exit 99 correctly failed the zero-failed-lifecycle gate.

An initial post-checkpoint API-loss test recorded one hangup 502. The custom shutdown signal branch could return without awaiting Actix's server drain. The corrected path disables competing signal handlers and polls graceful stop and server completion concurrently, with a 20-second drain inside Kubernetes' 30-second grace and five-second preStop. The passing rerun is retained alongside the real earlier failure. CNPG logs also showed about 40 seconds waiting for WAL receivers during primary deletion. The Minikube-only smart-shutdown interval changed from 45 to five seconds, while the 90-second stop/WAL budget and required synchronous replica remained unchanged. Recovery fell from 60.995 to 27.943 seconds and HTTP failures from 2.634% to 1.600%; all 1,461 acknowledged creates, turns and hangups survived. This does not establish zero-error failover or node-failure behaviour.

The final HPA rerun on the corrected image observed a desired increase after 71.3 seconds, an additional Ready pod after 89.8 seconds, and downscale after 371.2 seconds of cooldown. It still failed zero-failed-lifecycle and HTTP p95 below two seconds. Logs identified database-pool acquisition timeouts; the three-connection pool was not enlarged beyond the established HPA connection budget to manufacture a pass. Independent exact-ID SQL auditing verified all 3,418 acknowledged creates, 3,415 turns and 3,418 hangups, with no missing acknowledged data or pending checkpoints. Scaling correctness and data preservation do not imply sufficient sustained-load capacity.

An initial Groq smoke produced three replies but zero successful sentiment scores. After protocol changes, the ten-turn sample passed every sentiment/reply operation with mean stage durations 570.95/514.43 ms. Ten samples are insufficient for production p95/p99 claims. Isolated mock inference failure/recovery passed 34 lifecycles with bounded fallback and breaker recovery; mock latencies are not provider timings. Cross-pod alert testing observed the same nonempty alert after API replacement and exactly one SQL row.

Versioned raw artifacts and detailed acceptance definitions are linked in `docs/TEST04_AND_AI_VALIDATION.md` and `docs/PHASE_5_LOCAL_RESULTS.md`. Public Twilio voice/audio outcomes, sustained capacity, node/zone failure and backup restoration remain untested.

6. Discussion and Limitations

The principal reliability choice is versioned durable state rather than process-owned context or asynchronously replicated Redis alone. PostgreSQL checkpointing removes the observed acknowledged-session loss in the repeated Redis fault, but increases database dependence and latency. The 100-VU run fell from 2,142 to 1,160 lifecycles and HTTP p95 rose from 390.04 to 1,287.24 ms. These single runs are not an isolated causal capacity study. Database outages explicitly fail requests rather than fabricate commits. Neither this design nor the experiments prove cross-system linearizability; queue/session reconciliation and arbitrary partition behaviour need further testing.

The modular monolith reduces orchestration cost while sharing distributed dependencies. The roster is process-local; alerts are now durable and shared. Production needs roster persistence, checkpoint retention/privacy policy, projection reconciliation, least-privilege secrets, network policies, TLS and restoration tests. Readiness still depends on Redis and PostgreSQL. A single Kubernetes node cannot establish node/zone availability, and the PostgreSQL replacement test does not satisfy zero-failed-lifecycle availability.

Clinical and operational safety boundaries are equally important. Keyword risk scoring is not a substitute for a trained and independently evaluated risk model or a human crisis service. Synthetic sessions and deterministic test responses do not demonstrate safe real-caller outcomes. Any pilot would need a documented escalation protocol, consent and data-retention policy, privacy review for sensitive transcript storage, accessibility testing, and clear user messaging about the service's non-clinical scope. The current report therefore makes no claim that SayangCare is ready to handle real mental-health emergencies.

7. Conclusion and Future Work

SayangCare implements a voice-session prototype that combines signed Twilio webhooks, hot/cold persistence, recoverable session updates, distress-aware inference degradation, and priority-based human handoff. Its main contribution is the integration of user-context-sensitive degradation with state recovery and queue-backed escalation in a containerized distributed application. Workspace tests provide initial correctness evidence, including simulated Raft replication and leader loss; local single-node Minikube tests provide preliminary load and pod-replacement observations. Neither those results nor the current tests establish production capacity, live telephony reliability, or multi-node cluster availability.

Post-checkpoint API-loss testing now passes with awaited graceful shutdown; HPA scaling is measured but sustained-load acceptance still needs refinement. Remaining work is to reduce PostgreSQL failover interruption without weakening commit guarantees, qualify multi-node behaviour and backup restoration, and configure public TLS and production secret delivery. Live Twilio testing requires approval and consenting testers. Final submission also needs team-reviewed screenshots, requirements/page-limit verification and safety/retention review; neither synthetic success nor a small AI smoke establishes readiness for real mental-health emergencies.

References

[1] M. Fowler, “CircuitBreaker,” *martinfowler.com*, 2014. [Online]. Available: https://martinfowler.com/bliki/CircuitBreaker.html

[2] M. T. Nygard, *Release It!: Design and Deploy Production-Ready Software*, 2nd ed. Raleigh, NC, USA: Pragmatic Bookshelf, 2018.

[3] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *Proc. USENIX Annual Technical Conference (USENIX ATC ’14)*, 2014, pp. 305–319. [Online]. Available: https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro

[4] Redis Ltd., “High availability with Redis Sentinel.” [Online]. Available: https://redis.io/docs/latest/operate/oss_and_stack/management/senti

[5] Twilio, “Security: Validating requests.” [Online]. Available: https://www.twilio.com/docs/usage/security#validating-requests

[6] Kubernetes Authors, “Horizontal Pod Autoscaling.” [Online]. Available: https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/

[7] V. K. Jones, C. Yan, M. Y. Shade, et al., “Reducing loneliness and improving social support among older adults through different modalities of personal voice assistants,” *Geriatrics*, vol. 9, no. 2, p. 22, 2024, doi: 10.3390/geriatrics9020022.

[8] Y. Yang, C. Wang, X. Xiang, and R. An, “AI applications to reduce loneliness among older adults: A systematic review of effectiveness and technologies,” *Healthcare*, vol. 13, no. 5, p. 446, 2025, doi: 10.3390/healthcare13050446.